

Programming in R: Exercise 1

Felix Clouth, f.j.clouth@tilburguniversity.edu

Part 1: The Basics

If you have not done this yet, install R (<https://cran.r-project.org/>), and then the free version of RStudio (in that order; <https://www.rstudio.com/>).

1. Calculate $11 + 31$ by typing it into the R Console (the interactive pane, bottom left)
2. Try the following:

```
1+1
1 + 1
1 +    1
```

You see that R does not care about spaces, but that doesn't mean you shouldn't. Which of the lines is more readable?

3. Try a few more math expressions. **This time, write them down in the Source panel (top left) and execute them by either selecting the entire line, or by placing your cursor somewhere on the line and pressing Ctrl + Enter (Windows) or Cmd + Enter (Mac).** HINT: Some math operators '+', '-', '*', '/', '^'.
4. Try combining math expressions. Use parentheses to indicate the order of the math expressions (e.g., $(4 + 2) * 6$). Play around with sending commands to the Console line by line and by selecting multiple lines of code.

```
4 * 4 * 4
4 * (4 + 2)
```

5. Run the following lines:

```
1 + 1
2 * 6
```

Compare your results to what happens if you run the following lines:

```
1 + 1; 2 * 6
```

What's the difference? Which version do you prefer and why?

6. Run the following line:

```
25 / (6 - 1
```

What happened? How can you solve this?

7. Run the following line:

```
# 7 * 3
```

You'll see that the line of code does get copied to the Console, but you won't get any output. Whenever you start a line with the '#' symbol, R understands it is a *comment*. You use comments to explain what you are doing and why. This is useful for collaborators and future-you. Comments are not evaluated by R, so you can write whatever you want in comments, including R code. It is STRONGLY recommended that you make good use of comments. An example of how you can use comments is the following:

```
# Exercise 1.7
# Felix Clouth

# Load data -----
mydata <- read.table("mydata.txt")

# Analysis -----
# fit linear model
model1 <- lm(y ~ x, data = mydata)

# inspect results
summary(model1)
```

Note that this code won't run on your machines, because you don't have a file called "mydata.txt". This is merely meant as an illustration of comments.

From now on, I would like you to write all your code in .R scripts with a title and informative comments throughout.

8. A lot of the flexibility and functionality in R is due to all the different R packages you can install. To use an R package, you have to take the following two steps:

```
# Step 1: install.packages()
# this function downloads the package from the CRAN library to your computer
# you only have to do this once
install.packages("cowsay")

# Step 2: library()
# this function "activates" or "loads" the package into R, so that you can use it
# you have to do this each time you want to use the package
library("cowsay")

# now I can use the functions in the package cowsay:
say("I've successfully installed and used cowsay", by = "monkey")
```

There are a **lot** of R packages available online, so it can be tricky to find the one you need. A useful starting point is this page: <https://cran.r-project.org/web/views/>. Here, you can find many R packages organized by topic. Pick one topic that is relevant to you (e.g., Psychometrics, or Social Sciences), and install and load one of the packages. You only need to install a package once on your computer, but you need to activate it in every R session in which you want to use it.

Extra question

If you get bored, explore some other fun R packages, e.g., “fortunes”, “praise”, ...

Part 2: Data, Vectors, and Variables

9. Calculate the following:

- $\log(42)$, $\log(e)$, $\log(0)$, $\log(-1)$. What base is used for the log? Why do you get errors?
- $e^{3.73767}$. Why do you guess this is not exactly equal to 42?
- $\exp(\log(42))$. Is this exactly equal to 42?
- "harry" + 5. What is the result? Explain this result.

10. Run the following line: `a <- b <- ab <- ba <- 1`. What happened? How do you know (which commands were executed)? How many objects have been created?

11. Quit and save your workspace. Check whether your objects still exist when you start R again. How can you remove those objects from your workspace?

12. Evaluate:

```
fluffy <- 37  
fluffy + 5
```

- Why does the result differ from that of 9d?
- What type does the variable fluffy have?

13. Evaluate:

```
fluffy <- "An unfriendly animal?"  
fluffy
```

- Try evaluating "Fluffy". Then try "FLUFFY" and "f1UffY". All should give an error. Why?
- What type does the variable fluffy have? What would happen if you tried `fluffy + 5` now?

14. Evaluate:

```
gryffindor <- c("Harry", "Ron", "Hermione", "Neville", "Ginny", "Oliver")
```

- a. What type does the variable gryffindor have?
- b. What is the type of gryffindor if I do this:

```
gryffindor <- c("Harry", "Ron", "Hermione", 1, 2, 3)
```

15. Execute:

- a) Make a vector where you repeat the value 3, 12 times
- b) Make a vector where you repeat the pattern (2, 4, ..., 20), twice
- c) Make a vector where you repeat the value 1, 3 times and the value 4, twice
- d) Make a vector where you repeat each value of the pattern (2, 4, ..., 20), twice

The following two questions are taken from “A (very) short introduction to R” (Torfs & Brauer, 2014). The full document can be found on Canvas > Modules > General. Beside good practice material, this document also provides an excellent reference card.

- 16. Compute the difference between 2025 and the year you started at this university and divide this by the difference between 2025 and the year you were born. Multiply this with 100 to get the percentage of your life you have spent at this university. Use brackets if you need them.
- 17. Repeat the previous exercise but add several steps in between. You can give the variables any name you want, but the name has to start with a letter.
- 18. Say you want to buy a house and take out a mortgage of 500.000€ at 3.5% interest. You plan to pay back the mortgage in 30 years (360 monthly installments). What will be your monthly costs?

Part 3: Functions

- 19. A personality test was taken by a herd of llamas. The results for llamas' Agreeableness are below. Evaluate:

```
llama_agreeableness <- c(50, 20, 15, 80, 35, 52)
```

- a. What type does the variable llama_agreeableness have?
- b. What is the sum of the agreeableness scores of the llamas?
- c. What is the length of the vector?
- d. What is the average agreeableness of these llamas? (hint: divide sum by length).
- e. Use the function mean() to find the average.
- f. Find the standard deviation with the function sd().
- g. Make R show the llamas' scores expressed as number of standard deviations from the mean (sometimes called “z-scores”)

20. As you may know, agreeableness is supposed to increase slightly with one's age. That would probably make the older llamas more agreeable. Below are the llamas' ages:

Arthur	Zaphod	Trillian	Ford	Marvin	Rooster
14	9	10	25	11	15

- Create a vector with the llama's ages and assign it to a variable called `llama_age` (hint: see earlier questions on how to do this)
- There was a seventh llama, named Slartibartfast, who refused to cooperate with the personality assessment, generating a missing value (in R missing values are denoted as `NA`). Without evaluating it, what do you think this will do?

```
llama_age <- c(llama_age, NA)
```

- What about this: `llama_age <- c(llama_age, llama_age)`
- As you have probably figured out by now, the function `c()` takes vectors as arguments, and returns their combination as a new vector. It can also take named arguments, for example:

```
llama_age <- c("Arthur" = 14, "Zaphod" = 9)
```

Complete the new `llama_age` vector with names (i.e. as above but using all the names from the table). Evaluate `llama_age`. What is the difference with your definition of `llama_age` in question 20a in how they are displayed?

- What does `names(llama_age)` do?
21. Can you compute the mean of `llama_age` with outliers removed? Remove 20% of the extreme values of each end of the distribution. (Check the `help()` files of `mean()`.)
22. With the function `cor()` you can compute correlations. For example:

```
cor(c(1, 2, 3, 4), c(1, 4, 5, 15))
```

(Pay attention to the order of the parentheses and commas!)

- Is this correlation statistically significant ($\alpha = .05$)?
- What is the p-value? Which function can we use to find out? How can we find such a function?
- How can we find all functions that have "cor" in their name?

23. Check out the line of code below:

What does `%%` do? Can you ask help with `?%%`, as you can for `?mean`? If not, then how? Hint: try `31 %% 7` and `31 %/% 7`

```
```r
31 %% 7
```
```

24. Try the command “Set working directory” via the menu (under “Session”).

- What does it do?
- How could you do this without using the menu?
- Make a text file with `a <- 10; b <- 20; print(a*b)`. Run this file with the function `source()` and write the output to another file. How do you do this?

Part 4: Indexing

25. Evaluate:

```
a <- c(1, 2)
a[1] + 10
```

- What is the result?
- Why does the following code give an error?

```
a <- c(1, "hello")
a[1] + 10
```

26. If `x <- c(1, 3, 5, 7, NA)`, write an R expression that will return the output `1 3 5 7`.

27. For this question you need the vector `llama_age` that you created in question 20.

- Extract only the third llama’s age from the `llama_age` vector.
- Extract only the first two llamas’ ages from the `llama_age` vector.
- Extract only the first and fifth llamas’ ages from the `llama_age` vector.
- Extract all but the second to fourth llamas’ ages from the `llama_age` vector.
- When a vector has names, it is also possible to use the names as indices. For example, `llama_age["Marvin"]` should give 11. Extract Zaphod and Ford’s ages at the same time by using their names. The result should be a vector with values 9 and 25.

28. It turned out that Rooster was sensitive about his age and lied on the questionnaire. His real age is 16 instead of 15. Use indexing to change Rooster’s age to 16.

29. In this exercise you will calculate the t test statistic yourself in R to test whether the mean satisfaction with life (swl) scores of the two groups presented below differ significantly at an alpha significance level of .05:

Group 1: `c(11, 6, 13, 13, 5, 10, 14, NA, 13, 13, 9, 8, 4, 8, 8, 6, 17, 10, 5, 10)`

Group 2: `c("6", "1", "6", "6", "7", NA, "3", "2", "2", "5", "7", "3", "6", "1", "13",
"2", "2", "6", "8", "5")`

However, there is some messiness in the data that has to be fixed.

First, the data for group 2 is stored in characters (hint: search ?as.numeric).

Second, during data collection something went wrong for the data of group 2, and each even (at INDEX 2, 4, 6, etc.) participant's score was stored at half the intended value, set these values to their original intended value.

Third, both datasets have one missing response marked as NA.

To calculate the t test statistic for the comparison between group 1 and 2 use the following formula:

$$t.test.statistic = \frac{(mean_{sample1} - mean_{sample2})}{\sqrt{\frac{sd_{sample1}^2}{n_{sample1}} + \frac{sd_{sample2}^2}{n_{sample2}}}}$$

To calculate the df of this test use the following formula:

$$df = n_{sample1} + n_{sample2} - 2$$

To determine the critical t value for this test, use the following website:

<https://garyfisk.com/stats/TableT-testCriticalValue.html>

Tip: you can see whether your answer is correct by comparing it to the result of the build in `t.test()` function in R.

30. Read "A (very) short introduction to R" (Torfs & Brauer, 2014) up until section 6.1 (Vectors), and make the accompanying exercises. The full document can be found on Canvas > Modules > General. There will be some repetition of material you practiced in Part 2 and 3 as well.
